**UCS503P – Software Engineering**

# Vigilante

A Personalized Email Prioritization System

*Prototype II — The Personalization Layer*

*A Sender-Aware Contextual Bandit with Online Adaptation*


**Submitted by:**

1024030679 Aryan Sharma
1024030687 Maithli Arora
1024030676 Tushar Sharma

Group: 3C5A
BE Third Year, CSE


**Submitted to:**

Dr. Raghav B. Venkataramaiyer

Prototype II Evaluation


**Computer Science and Engineering Department**

Thapar Institute of Engineering and Technology, Patiala

# Contents


**1 Introduction** **2**
- 1.1 Background and Context . . . 2
- 1.2 Recap of Prototype I . . . 2
- 1.3 Motivation: Limitations of the Baseline . . . 2
- 1.4 Objectives of Prototype II . . . 3

**2 Methodology and Design** **4**
- 2.1 System Overview . . . 4
- 2.2 The Feedback Infrastructure . . . 4
- 2.3 Exploration: Resolving Feedback Starvation . . . 5
- 2.4 The Personalization Layer: A Contextual Bandit . . . 5
  - 2.4.1 Formulation . . . 5
  - 2.4.2 Sender-Aware Context and Per-Sender Memory . . . 5
  - 2.4.3 Adaptation to Drift . . . 6
- 2.5 Safe Deployment: Shadow Mode . . . 6
- 2.6 Technical Stack . . . 6

**3 Results and Evaluation** **7**
- 3.1 Evaluation Methodology . . . 7
- 3.2 Honest Measurement of the Baseline . . . 7
- 3.3 Validation of the Personalization Mechanism . . . 8
- 3.4 Current Data Status . . . 8
- 3.5 Go-Live Criteria: Measurement and Flip . . . 8

**4 Conclusion** **10**
- 4.1 Summary of Work . . . 10
- 4.2 Key Findings . . . 10
- 4.3 Future Work and Recommendations . . . 10
- 4.4 Final Remarks . . . 11

Chapter 1

# Introduction

## 1.1 Background and Context

Vigilante is a system that connects to a user's inbox, judges the importance of each incoming email with a machine-learning model, surfaces the messages that matter, and reversibly removes clearly low-value mail — all without continuous user intervention. The first prototype established this capability with a single, global importance classifier deployed autonomously in the cloud. The present report concerns the second prototype, whose purpose is to move the system from a generic judgement of importance to a *personalized* one: a layer that learns what each individual user considers important, from that user's own behaviour, and adapts as their priorities change.

The motivating observation is that importance is not universal. The same email — an announcement from a particular sender, a newsletter, a recruiter's message — can be critical to one user and irrelevant to another. A single model trained on pooled data captures the *average* notion of importance, but cannot capture any individual's. Prototype II addresses this gap directly.

## 1.2 Recap of Prototype I

The first prototype delivered a complete, deployed baseline. A logistic-regression classifier was trained on 6,000 human-verified emails using TF–IDF features of the subject (500) and snippet (500) together with twelve engineered signals, and achieved a ROC–AUC of 0.949 on a held-out test set. Around this model, a full operational pipeline was built: a persistent Gmail connection via OAuth 2.0 with a stored refresh token; a background processor that uses the Gmail History API with per-user checkpoints to fetch only new mail; reversible archiving by means of a dedicated `Vigilante/Archived` label; and autonomous cloud execution on AWS Lambda triggered by Amazon EventBridge every five minutes. This baseline is the foundation on which Prototype II builds, and it continues to run unchanged as the system's cold-start layer.

## 1.3 Motivation: Limitations of the Baseline

Four limitations of the global baseline define the scope of Prototype II:

1. **Distribution mismatch.** The 0.949 ROC–AUC is an offline figure on a curated dataset; it need not hold on the live, heterogeneous inboxes of real users.

2. **No personalization.** One model scores every user identically and cannot adapt to an individual's notion of importance.

3. **Feedback starvation.** Because the system archives low-scoring mail, the user never sees those messages and never acts on them; the system therefore cannot learn whether hiding them was correct. It becomes most confident precisely where it is blind.

4. **No adaptation.** A user's priorities drift over time, but a frozen model cannot track that change.

## 1.4 Objectives of Prototype II

1. **Instrument honest feedback.** Record, for every scored email, the model's prediction together with the signals needed for personalization, and derive a truthful importance label from the user's subsequent behaviour.

2. **Evaluate the baseline honestly on live data** and diagnose the true obstacle to personalization rather than assuming one.

3. **Build a per-user personalization layer** — a sender-aware contextual bandit — that adjusts the baseline's score to each individual, and that adapts as their taste changes.

4. **Deploy it safely.** Introduce the learning layer without any risk to the proven baseline, and define objective, data-driven criteria for when it may be allowed to take over decisions.

Chapter 2

# Methodology and Design

## 2.1 System Overview

Vigilante was designed from the outset as a two-layer intelligence system: a global baseline classifier for cold-start judgement, and a personalization component that adapts the baseline to each user. Prototype I implemented the first layer; Prototype II implements the second. The personalization layer does not replace the baseline — it sits on top of it, taking the global score as a prior and adjusting it per user. New users, and users with no history for a given sender, fall back cleanly to the baseline.

The work of Prototype II proceeded along a deliberate path, each step motivated by the finding of the previous one: (i) instrument feedback; (ii) measure the baseline honestly; (iii) on finding that the true obstacle was a shortage of honest data rather than a weak model, introduce *exploration* to generate that data; (iv) build the personalization bandit; (v) add drift adaptation; and (vi) deploy the whole learning layer in a risk-free *shadow mode*.

## 2.2 The Feedback Infrastructure

Personalization requires ground truth about what each user actually finds important. Prototype II introduces a feedback pipeline that produces this ground truth from behaviour rather than from manual labelling.

**Prediction logging.** At scoring time, every email produces a feedback record containing the model's score, the decision taken (kept or archived), the normalized sender address, the Gmail category, and the context vector later used for personalization (Section 2.5). Storing the context at scoring time ensures that learning uses exactly the inputs that drove the decision.

**Label derivation.** A reconciliation pass runs on each cycle. For feedback records older than twenty-four hours — a delay that gives the user time to act — the system re-reads the message's current Gmail labels and derives a truthful importance label from the user's behaviour, as summarised in Table 2.1.

| User behaviour | Interpretation | Label |
|---|---|---|
| Read, starred, or un-archived | engaged with it | important (1) |
| Archived by the user, or trashed | saw it and dismissed it | not important (0) |
| Kept in inbox but never opened | ambiguous (busy $\neq$ unimportant) | unlabelled |

Table 2.1: Derivation of honest labels from user behaviour. Ambiguous cases are deliberately left unlabelled rather than assumed.

**Honest signals only.** One behaviour is deliberately excluded from learning: an email that the system archived and that the user never acted upon. Such a record labels itself "not important"

only because the system hid it — the user never had the opportunity to judge it. These self-confirming records are recorded but never used to train the model or the bandit, for reasons made concrete in Section 3.2.

## 2.3 Exploration: Resolving Feedback Starvation

The honest evaluation of the baseline (Chapter 3) established that the binding obstacle to personalization was not model quality but a severe shortage of honest *negative* signal: the system archived borderline mail so aggressively that users were rarely given the chance to dismiss anything themselves. To generate this missing signal, an exploration mechanism was introduced into the archiving decision.

For an email whose score falls in a borderline band (between the exploration floor and the archive threshold), the system retains the message in the inbox — rather than archiving it — with a fixed probability. The user is then able to judge it, producing a genuine negative (if dismissed) or positive (if engaged with). The high-confidence junk below the band is still archived, so the cost in inbox clutter is bounded. This exploration is not merely a data-collection device: the explicit balancing of exploiting current knowledge against exploring uncertain cases is the defining behaviour of the contextual bandit that follows, so the mechanism is a constituent part of the personalization layer rather than a temporary patch.

## 2.4 The Personalization Layer: A Contextual Bandit

### 2.4.1 Formulation

The personalization layer is a per-user contextual bandit based on the LinUCB (Linear Upper Confidence Bound) algorithm. For each user the bandit maintains a $d \times d$ matrix $A$ and a $d$-dimensional vector $b$, initialised as $A = I_d$ and $b = \mathbf{0}$. Given a context vector $x \in \mathbb{R}^d$ for an email, the learned weight vector and the personalized importance estimate are

$$\theta = A^{-1}b, \qquad \hat{p}(x) = \theta^\top x. \tag{2.1}$$

To balance exploitation against exploration, the bandit can instead score an email by its upper confidence bound, which adds a bonus proportional to the model's uncertainty about that context:

$$p_{\mathrm{UCB}}(x) = \theta^\top x + \alpha\sqrt{x^\top A^{-1}x}\,, \tag{2.2}$$

where $\alpha \geq 0$ controls the degree of exploration. When a feedback record resolves to a reward $r \in \{0, 1\}$, the bandit learns by the rank-one update

$$A \leftarrow A + xx^\top, \qquad b \leftarrow b + r\,x. \tag{2.3}$$

This update is online: the bandit improves continuously with every resolved interaction, with no batch retraining step.

### 2.4.2 Sender-Aware Context and Per-Sender Memory

The context vector is what makes the bandit personalize on *who* sent an email rather than merely rescaling the global score. A low-dimensional, sender-aware context was chosen deliberately: a high-dimensional one-hot encoding over individual senders would be far too sparse to learn from the modest data available. Instead, specific-sender preference is captured through a compact engagement statistic. The context used is

$$x = \big[\,1,\ s,\ e,\ v,\ u\,\big]^\top, \tag{2.4}$$

where $s$ is the global model's score, $e$ is the user's engagement with this email's sender, $v \in \{0, 1\}$ indicates whether the user has prior history with the sender, and $u \in \{0, 1\}$ indicates bulk mail (a promotional or social category, or the presence of an unsubscribe header). The constant 1 is a bias term.

The engagement feature $e$ is supplied by a per-user, per-sender memory that records, for each (user, sender) pair, the number of interactions *seen*, the number the user *engaged* with, and the number *dismissed*. The engagement rate is reported with Laplace smoothing so that a sender with little history is treated neutrally:

$$e = \frac{\text{engaged} + 1}{\text{seen} + 2}, \qquad e = 0.5 \text{ when the sender is unseen.} \tag{2.5}$$

Through this memory, the bandit learns per-user weights such as a strong positive weight on $e$ for a user who cares about specific known senders, or a negative weight on $u$ for a user who dismisses bulk mail — while still generalising to unseen senders through $s$ and $u$.

### 2.4.3 Adaptation to Drift

A user's priorities change over time, and a learner that weights all past evidence equally adapts to such change only very slowly. To track drift, both the bandit and the sender memory apply exponential forgetting. The bandit's update is discounted by a factor $\gamma \in (0, 1]$,

$$A \leftarrow \gamma A + xx^\top, \qquad b \leftarrow \gamma b + r\,x, \tag{2.6}$$

so that the influence of an observation decays geometrically with each subsequent update. The sender memory is decayed analogously before each new interaction is added,

$$\text{seen} \leftarrow \gamma_s \,\text{seen} + 1, \quad \text{engaged} \leftarrow \gamma_s \,\text{engaged} + \mathbf{1}[r = 1], \quad \text{dismissed} \leftarrow \gamma_s \,\text{dismissed} + \mathbf{1}[r = 0]. \tag{2.7}$$

The discount factors act as a single, interpretable control over the memory horizon: a value close to 1 yields long memory and stability, a lower value yields faster adaptation. Conservative defaults of $\gamma = 0.999$ for the bandit and $\gamma_s = 0.98$ for the sender memory are used, to be tuned once sufficient longitudinal data is available to observe drift directly.

## 2.5 Safe Deployment: Shadow Mode

The learning layer is deployed in *shadow mode*. The bandit and the sender memory are updated on every honest feedback record, so they learn from live behaviour continuously; but they do not drive any decision. The proven global baseline continues to make every archiving decision. This separation guarantees that a partially-trained bandit cannot hide a user's important mail, because it is not in the decision path. The learning layer thus accrues value from the first day of deployment at zero risk, and is promoted to making decisions only after it has demonstrably earned that role by the criteria in Section 3.5.

## 2.6 Technical Stack

Prototype II is implemented within the existing architecture and adds no external dependencies. The bandit's linear algebra (matrix inversion, rank-one updates) is implemented directly in Node.js so that it runs inside the existing serverless background processor. Two new MongoDB collections are introduced — one storing each user's bandit state ($A$, $b$), and one storing the per-user, per-sender engagement memory — alongside the extended feedback collection. The sender, category, and unsubscribe signals required by the context vector are already produced by the existing Gmail-access module and required no change to that module.

Chapter 3

# Results and Evaluation

## 3.1 Evaluation Methodology

Prototype II was evaluated along three axes: an honest measurement of the baseline on live user behaviour; a controlled validation of the personalization and drift-adaptation mechanisms; and the definition of objective criteria governing when the bandit may take over decisions. Because the personalization layer learns online from live data that is still accumulating, the evaluation distinguishes carefully between what can be measured now and what is gated on further data.

## 3.2 Honest Measurement of the Baseline

The baseline was measured directly on live behaviour by comparing each email's stored prediction against the truthful label derived from the user's action. The naive result was a live ROC–AUC of 0.992 — higher than the offline 0.949, and implausibly so. Investigation of the label composition explained why.

| Outcome | Count | Signal quality |
|---|---|---|
| Archived, never seen (self-confirmed) | 298 | confounded |
| Read | 177 | honest positive |
| User-archived | 5 | honest negative |
| Un-archived | 2 | honest positive |

Table 3.1: Composition of resolved feedback. Of 482 resolved records, 298 negatives were self-confirming, leaving only 5 honest negatives.

As Table 3.1 shows, the overwhelming majority of the negative labels were *self-confirming*: emails the system had archived and the user never saw. Such a record is labelled "not important" as a direct consequence of the system's own decision, so the model appears to predict it correctly by construction. This circularity inflates the measured AUC toward unity regardless of true performance.

Recomputing the metric on honest signals only — excluding the self-confirming records — gave a ROC–AUC of 0.838. This figure, however, rested on only five honest negative examples, a sample far too small to support a stable estimate. The decisive finding was therefore not either AUC value but the composition itself: the system possessed almost no honest negative signal. The obstacle to personalization was a *data-generation* problem, not a modelling problem — a direct consequence of the feedback-starvation limitation — and this finding is what motivated the exploration mechanism of Section 2.3. That the inflated 0.99 was identified and rejected, rather than reported, is itself regarded as a substantive result of the evaluation.

## 3.3 Validation of the Personalization Mechanism

Because live personalization is gated on the accumulation of honest data, the personalization and drift mechanisms were validated in a controlled simulation that exercises the algorithm directly.

**Sender personalization.** A bandit was trained on a simulated user whose true notion of importance depended on sender engagement rather than on the global score. With the global score held fixed at a deliberately low value of 0.2, the bandit's predicted importance was then observed as the sender-engagement feature was varied. The results (Table 3.2) confirm that the bandit learns to raise importance for senders the user engages with, independently of the global score — that is, it personalizes on who sent the email.

| Sender engagement | Predicted importance (global score fixed at 0.2) |
|---|---|
| 0.1 | 0.00 |
| 0.3 | 0.12 |
| 0.5 | 0.40 |
| 0.7 | 0.68 |
| 0.9 | 0.96 |

Table 3.2: Personalization validated in simulation: at a fixed low global score, predicted importance rises with the user's engagement with the sender.

**Drift adaptation.** The forgetting mechanism was validated against the following scenario. A sender the user formerly engaged with over many interactions develops a high engagement rate, so the bandit boosts its mail. The user then stops engaging and begins dismissing that sender's mail. Without forgetting, the large accumulated history of engagement outweighs the few recent dismissals, and the sender continues to be boosted for a long time. With the discounted update, the old evidence decays with each dismissal, and the engagement rate crosses below the decision boundary within roughly ten to fifteen dismissals, after which the sender's mail is once again archived. The mechanism thus tracks a genuine change in user preference, satisfying the adaptation objective.

## 3.4 Current Data Status

At the time of evaluation the feedback collection held approximately 482 resolved records, of which about 37% were positive — a healthy class balance. The binding constraint remained the honest negative signal: only five user-driven negatives had been collected prior to the introduction of exploration. With exploration now deployed, honest negatives are expected to accumulate over a period of roughly two weeks of normal use, after which the personalization layer can be measured meaningfully. The rate of accumulation is contingent on connected accounts remaining authorised, a platform constraint discussed in Section 4.3.

## 3.5 Go-Live Criteria: Measurement and Flip

Objective, data-driven criteria govern the promotion of the bandit from shadow mode to active decision-making.

**Measurement.** The bandit and the baseline are compared on live behaviour using a prequential (test-then-train) protocol: processing resolved records in chronological order, each record is first scored by the bandit in its current state — before that record is used to update it — so that no record is ever evaluated after it has been learned from. The ROC–AUC of the bandit's

predictions is then compared against that of the baseline's scores, both overall and per user. Personalization gains are expected to manifest at the per-user level.

**Flip.** Only if the bandit beats the baseline by a meaningful margin is it promoted. The promotion itself substitutes the bandit's personalized score for the global score in the archiving decision, subject to safety rails: a high-confidence-important email is never hidden regardless of the bandit; a user with insufficient history falls back to the baseline; and a configuration switch permits the behaviour to be disabled instantly. Promotion is therefore reversible and governed by evidence, not by a schedule.

Chapter 4

# Conclusion

## 4.1 Summary of Work

Prototype II extends Vigilante from a global importance classifier to a personalized, adaptive system. A feedback infrastructure was built that derives truthful importance labels from user behaviour and records the signals required for personalization. An honest measurement of the baseline on live data exposed an inflated metric and diagnosed the true obstacle — a shortage of honest negative signal caused by aggressive archiving — which an exploration mechanism now addresses. A per-user, sender-aware contextual bandit was designed and implemented, taking the global score as a prior and learning each user's preference for particular senders and types of mail through a compact engagement memory; exponential forgetting was added so that the system tracks changing priorities. The entire learning layer was deployed in shadow mode, where it learns from live behaviour at zero risk to the proven baseline, and objective criteria were defined for its eventual promotion to decision-making.

## 4.2 Key Findings

- **Finding 1:** A headline metric can be an artefact of the system that produced it. The live ROC–AUC of 0.99 was inflated by self-confirming labels; identifying and rejecting it, rather than reporting it, was essential to progress.
- **Finding 2:** The obstacle to personalization was a data-generation problem, not a modelling one. Aggressive archiving starved the system of honest negative signal, which reordered the work — exploration had to precede any attempt to learn.
- **Finding 3:** Sender-level personalization is achievable on modest data without a high-dimensional sender encoding, by summarising a user's history with each sender in a single smoothed engagement statistic.
- **Finding 4:** Online learning with exponential forgetting addresses personalization and adaptation together: the same mechanism that learns a user's taste also unlearns it when the taste changes.

## 4.3 Future Work and Recommendations

1. **Promotion to active personalization.** Once sufficient honest data has accumulated, apply the measurement protocol of Section 3.5 and, if the bandit prevails, promote it from shadow mode under the stated safety rails.
2. **Richer context.** Extend the context vector with further cheap, generalisable signals (for example, time-of-day patterns or whether the user has previously replied to the sender) as data volume permits.

3. **Persistent authorisation.** Resolve the platform's testing-mode token expiry by publishing the application, so that connected accounts remain authorised without periodic re-connection and the flow of learning data is uninterrupted.

4. **Tuning the memory horizon.** Once longitudinal data exhibits genuine drift, calibrate the forgetting factors $\gamma$ and $\gamma_s$ against observed rates of change.

## 4.4 Final Remarks

Prototype II demonstrates a complete, deployed personalization layer that learns each user's notion of importance from behaviour and adapts as that notion changes, built safely on top of a proven baseline. As with the first prototype, the work reaffirmed the value of honest engineering: rejecting an inflated metric rather than reporting it, diagnosing the real obstacle before building a solution, choosing a model proportionate to the available data, and deploying a learning system in a mode that cannot harm the user before it has earned trust. These principles shaped both the design of the personalization layer and the team's understanding of how to build adaptive systems responsibly.